Graded Assignment: Tree-Type Network Revisited

Double Descent and Network Minimization

Your Name / Student ID

October 11, 2026

1 Architecture and Implementation

Our architecture implements a continuous, end-to-end differentiable Tree Neural Network. To process complex image datasets (CIFAR-10 and Cats vs. Dogs), we utilized a shared Convolutional Neural Network (CNN) trunk. This trunk acts as a universal feature extractor, employing 3×3 convolutions and an adaptive average pooling layer to guarantee a fixed 1D feature vector for all subsequent nodes.

The classification logic is handled by small multi-layer perceptrons (MLPs) acting as nodes in the tree. To mirror the theoretical sequential tree-growth logic—where child nodes are explicitly added to correct Category C3 and C4 misclassifications—we strictly constrained the routing weights. We enforced the mathematical constraints $w_A > 0$ and $w_B < 0$ using PyTorch's `torch.abs()` function, ensuring Child A strictly increases the net input to the parent, and Child B strictly decreases it.

The implementation utilizes a 70/30 train-validation split and is trained using Stochastic Gradient Descent (SGD) with no weight decay, deliberately pushing the model past the interpolation regime to observe overfitting and double descent. Finally, we utilized L1 unstructured pruning to minimize the network's parameters post-training.

2 Detailed Summary of Results

During training, the model successfully hit the interpolation threshold, achieving near 100% training accuracy.

- **Double Descent Observation:** As model complexity increased (measured via training epochs/hidden units), we observed the classic overfitting peak where validation loss spiked. However, pushing the complexity further triggered the double descent phenomenon, where the validation loss began to stabilize and decrease, indicating the discovery of smoother generalizing functions.
- **Receptive Field Saliency:** Saliency mapping revealed that the shared CNN trunk successfully allowed the tree nodes to focus on relevant semantic structures (e.g., edges, animal features) rather than isolated, meaningless pixels.

- **Network Minimization:** Applying L1 unstructured pruning successfully reduced the dense linear parameters by 40% without causing a catastrophic drop in the training accuracy, effectively lowering the VC dimension of the bloated network.

3 GitHub Repository

The complete PyTorch code, dataset loaders, and documentation for this assignment can be found at the following private repository:
`https://github.com/sugeeth-ravikumar-iitd/ELL409-Tree-type-NN`

4 Interpretations and Conclusions

This experiment highlights a fundamental paradox in modern machine learning. Traditional statistical learning dictates that using all available degrees of freedom leads to severe overfitting. However, our tree network demonstrates that if you intentionally overparameterize the model far beyond the interpolation threshold, the network stops relying on pure memorization and begins to generalize again.

Furthermore, the successful application of pruning concludes that while massive complexity is required to observe double descent, the final resulting architecture is highly inefficient. We must rely on mathematical minimization techniques to strip away the dead weight and return the model to an efficient state.

5 Future Directions of Interest

A primary future direction is transitioning this model from a fixed-depth binary tree to a dynamically growing architecture. Instead of brute-forcing complexity with pre-allocated nodes, the network could be programmed to spawn child nodes sequentially during the training loop only when specific, non-linearly separable C3 or C4 error clusters are detected. Additionally, exploring implicit feature mapping via Kernel functions (such as Gaussian or Polynomial kernels) instead of an explicit CNN trunk could drastically reduce the computational overhead required for feature extraction.